

2025-1

Basic Algorithm Study For ■ Beginners

기초 알고리즘 스터디 6주차

01

스택

· 정의

02

큐

· 정의

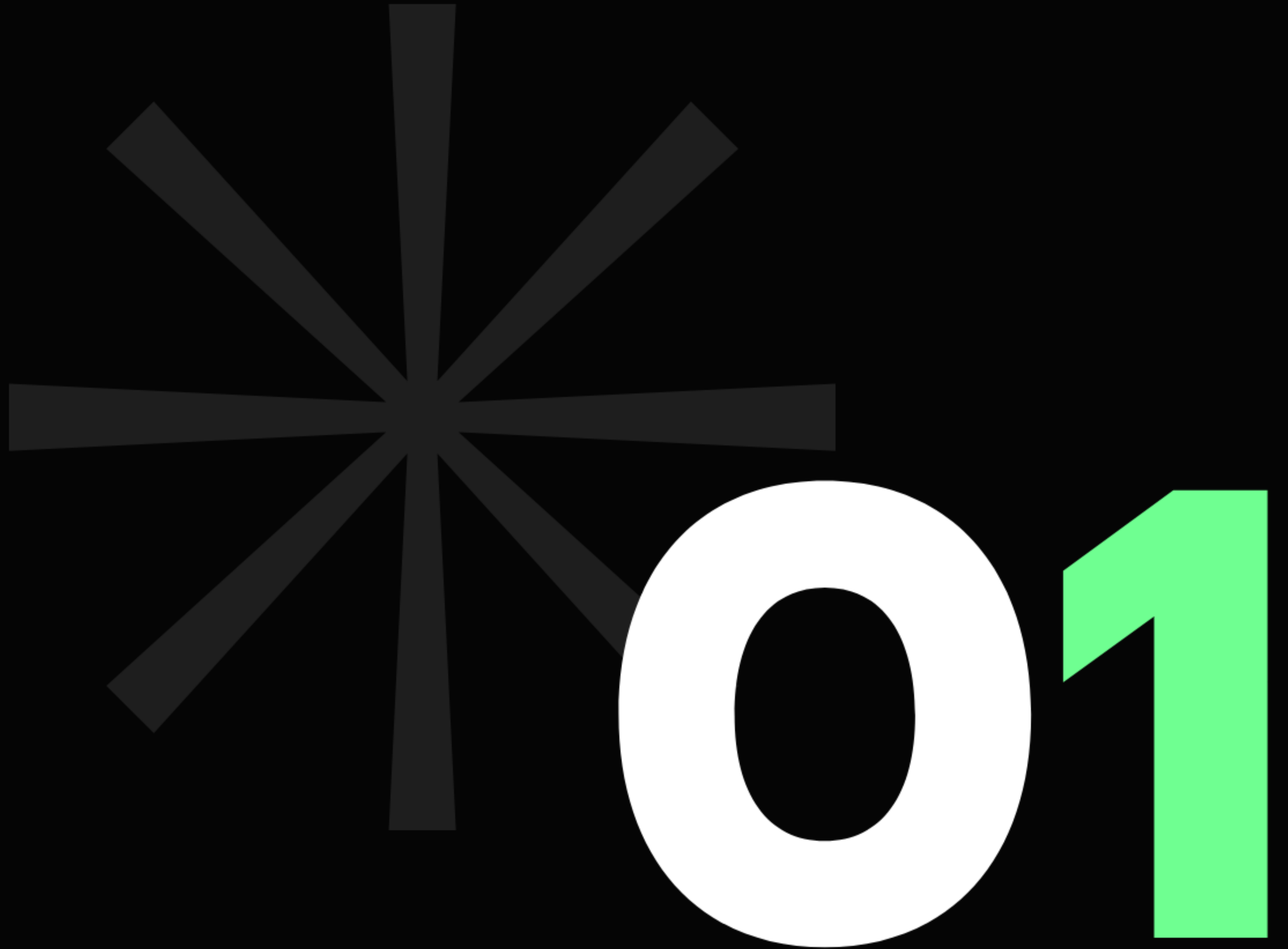
03

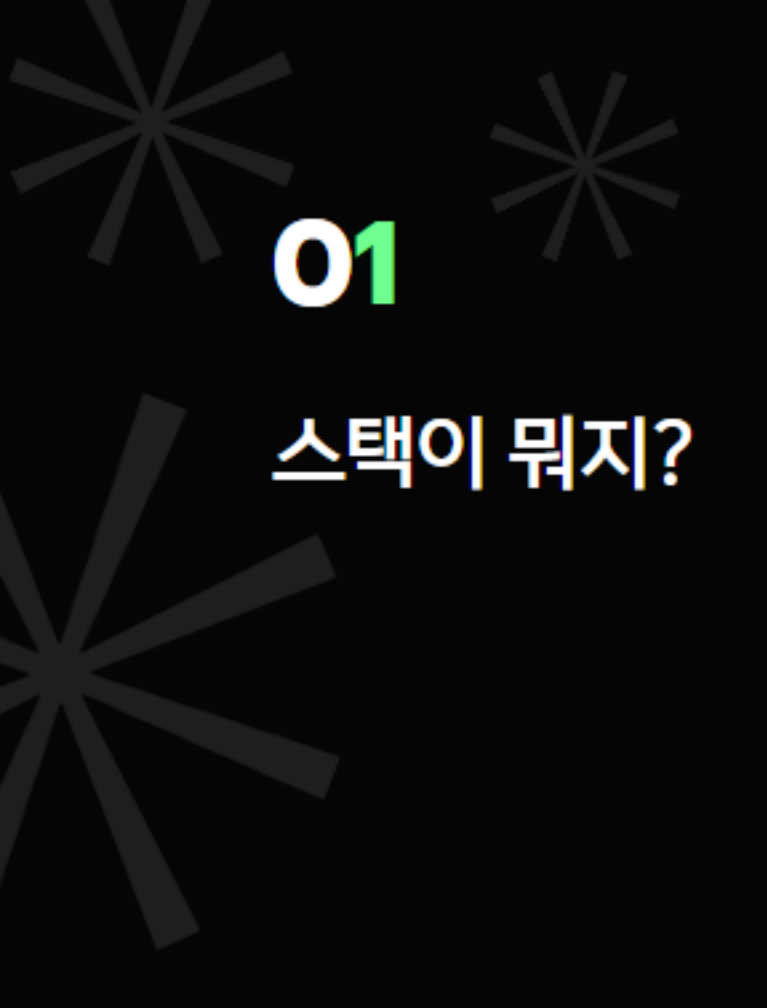
우선순위 큐

· 작동 원리?

스택

—





01

스택이 뭐지?

오늘은 굉장히 이론 위주의 수업입니다..
그래도 나중에 다 피가 되고 살이 된다는 느낌으로 들어주십사...



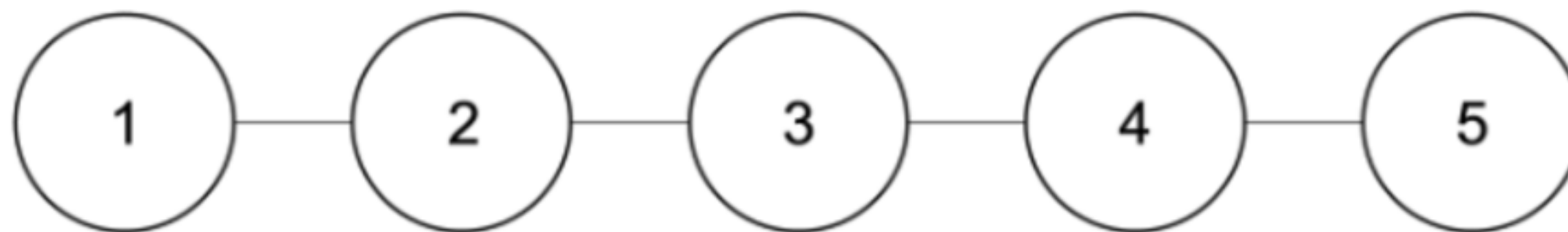
01

스택이 뭐지?

그래서 지금부터 배워볼 것은
스택과 큐, 우선순위 큐라는 세가지 자료구조입니다.

이 중에 스택과 큐는 **선형 자료구조** 라고 하는데,
선형 자료구조의 특징으로는

- **순차적**으로 연결됨
- 원소들이 **연속적인 메모리 공간**에 배치되었거나
포인터로 연결되어 있음

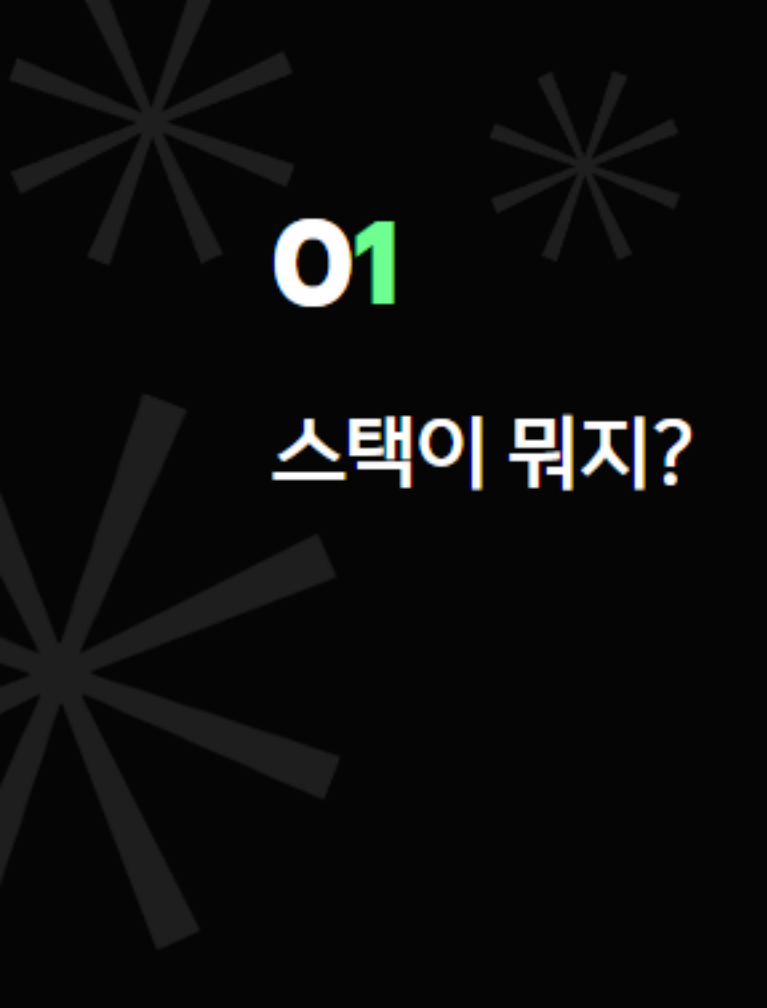




01

스택이 뭐지?

어쨌든 스택의 특징은
선형 자료구조이며,
연속적인 메모리 공간을 사용합니다.
LIFO (Last In, First Out) 구조를 갖고 있습니다.

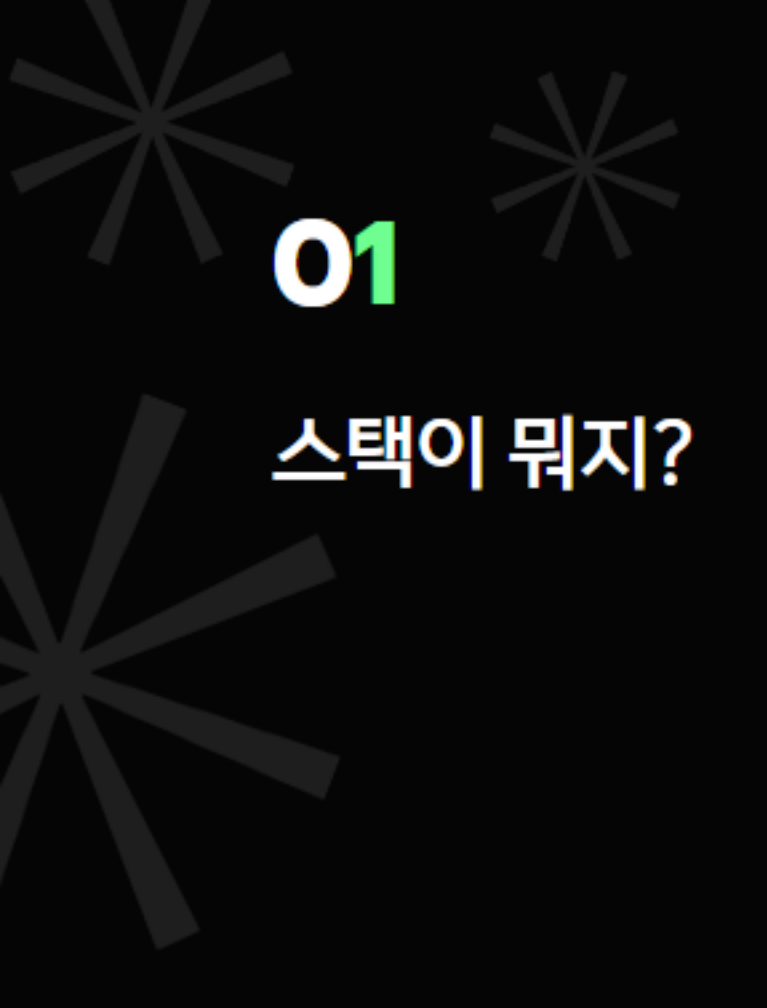


01

스택이 뭐지?

비유해보자면...

가방 안에 차곡차곡 짐을 쌓아서 넣은 상황과 비슷합니다.
물건을 꺼내기 위해서는 맨 위의 물건부터 빼야겠죠?



01

스택이 뭐지?

LIFO (Last In, First Out) 구조란?

가장 마지막에 삽입된 원소가 가장 먼저 제거됨

`stack.append()`

`stack.pop()`

메소드를 사용했을 때 무슨 일이 일어날까요?



01

스택이 뭐지?

`st = []`

`st.append(3)`

`st.append(4) # st = [3, 4]`

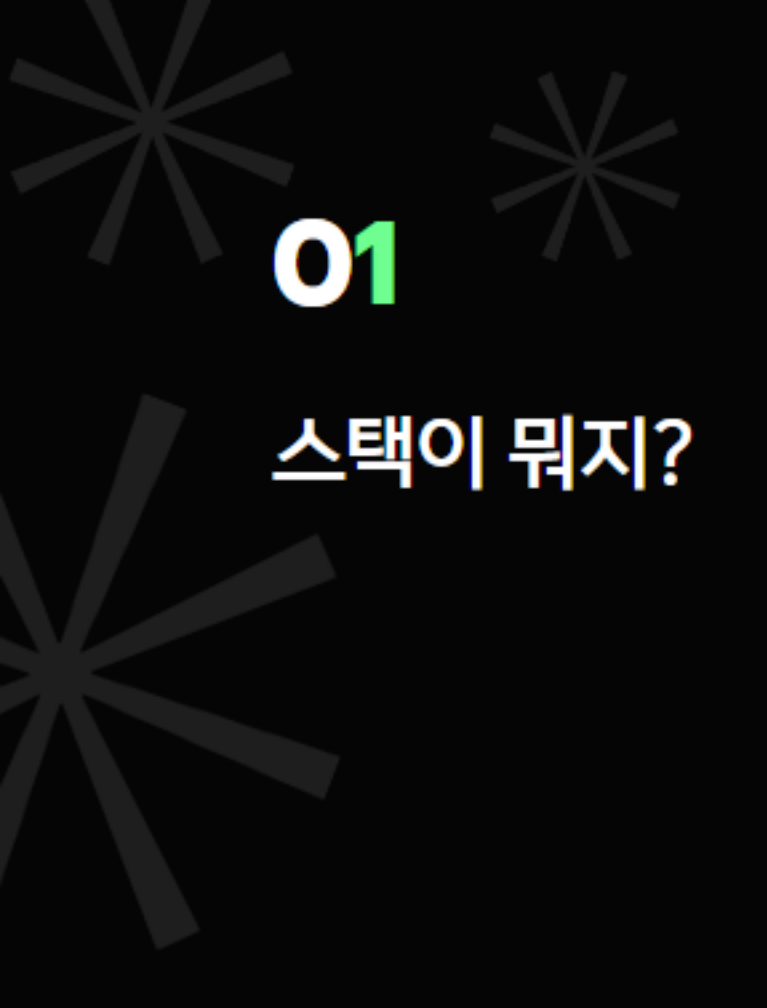
`st.pop() # st = [3]`



01

스택이 뭐지?

잠깐, 어디서 많이 본 것 같은데
말만 거창하지 그냥 리스트잖아요!



01

스택이 뭐지?

맞습니다. 그냥 리스트입니다.

파이썬의 리스트는 기본적으로 스택과 같은 방식으로 동작합니다.

그래서 스택이 필요할 때는 그냥 리스트를 사용하시면 됩니다.



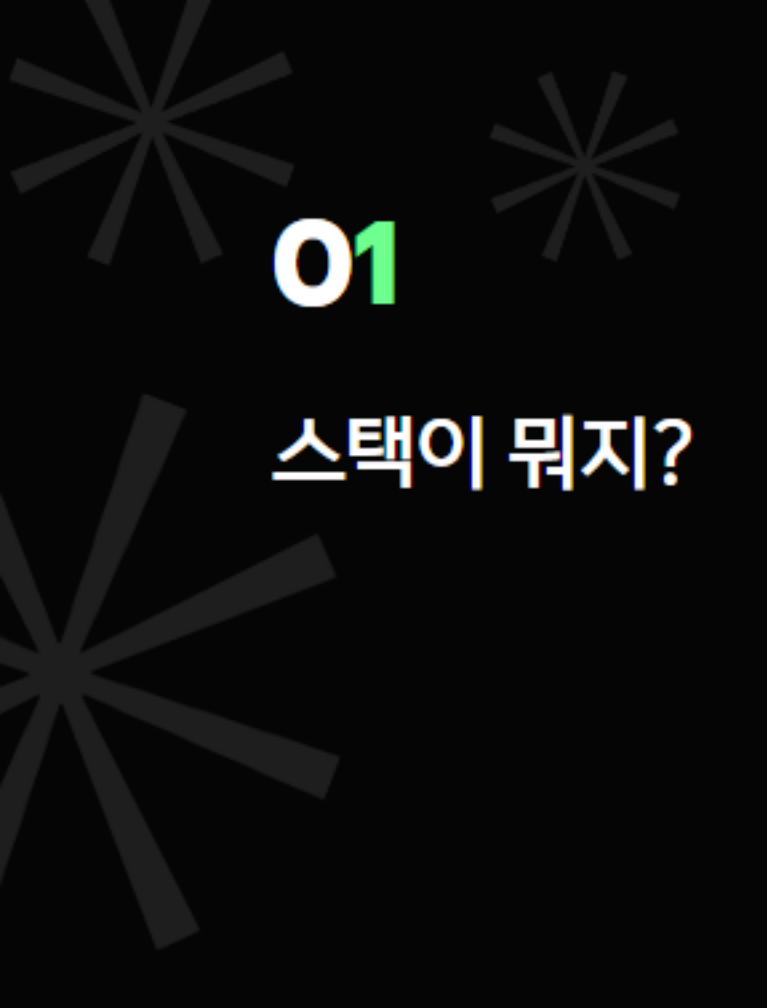
01

스택이 뭐지?

흠.. 근데 뭔가 단어가 익숙한데요?

스택... 스택...

함수 스택?



01

스택이 뭐지?

그렇습니다.

2주차 재귀함수 보충 자료에 있었던 함수 스택을 기억하시나요?

함수 호출과 스택은 깊은 연관이 있습니다.

스택이 뭐지?

print: ABABADCC

```
def recur(0, 2):
```

```
    line(i, string: "A")
```

```
    if i == end:
```

```
        line(i, string: "D")
```

```
        line(i, string: "C")
```

```
    return
```

```
2 line(i, string: "B")
```

```
3 recur(i+1, end)
```

```
12 line(i, string: "C")
```

```
def recur(1, 2):
```

```
4 line(i, string: "A")
```

```
    if i == end:
```

```
        line(i, string: "D")
```

```
        line(i, string: "C")
```

```
    return
```

```
5 line(i, string: "B")
```

```
6 recur(i+1, end)
```

```
11 line(i, string: "C")
```

```
def recur(2, 2):
```

```
7 line(i, string: "A")
```

```
    if i == end:
```

```
8 line(i, string: "D")
```

```
9 line(i, string: "C")
```

```
10 return
```

```
line(i, string: "B")
```

```
recur(i+1, end)
```

```
line(i, string: "C")
```

함수 스택



호출 트리





01

스택이 뭐지?

함수를 호출하는 과정이 .append()
함수가 끝나는 과정이 .pop()
호출된 함수를 **스택에 들어간 원소**라고 가정하면
딱 들어맞죠?



01

스택이 뭐지?

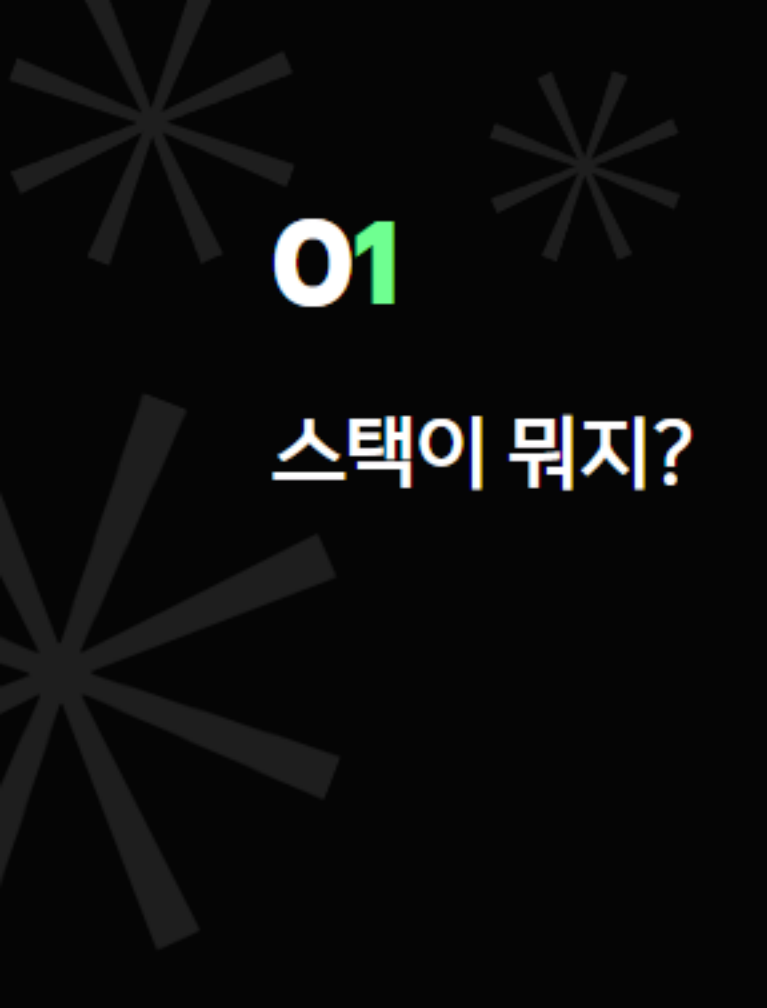
갑자기 이 얘기를 왜 하나면..
8주차에 배울 그래프 탐색과 관련이 있기 때문입니다!
꼭 잊지 말고 기억해주세요!



01

스택이 뭐지?

그럼 이제 연속된 메모리 공간이라는 게
무슨 말인지 알아보시다.



01

스택이 뭐지?

컴퓨터 메모리는 각각의 바이트마다 특정한 주소를 갖고 있습니다.
편의를 위해 스택의 첫번째 원소가 가진 주소를 0이라고 해 봅시다.



01

스택이 뭐지?

st = [3]

주소 -> 값

0 -> 3



01

스택이 뭐지?

`st = [3, 4]`

주소 $\rightarrow$ 값

0 $\rightarrow$ 3

1 $\rightarrow$ 4

01

스택이 뭐지?

$st = [3, 4, 5]$

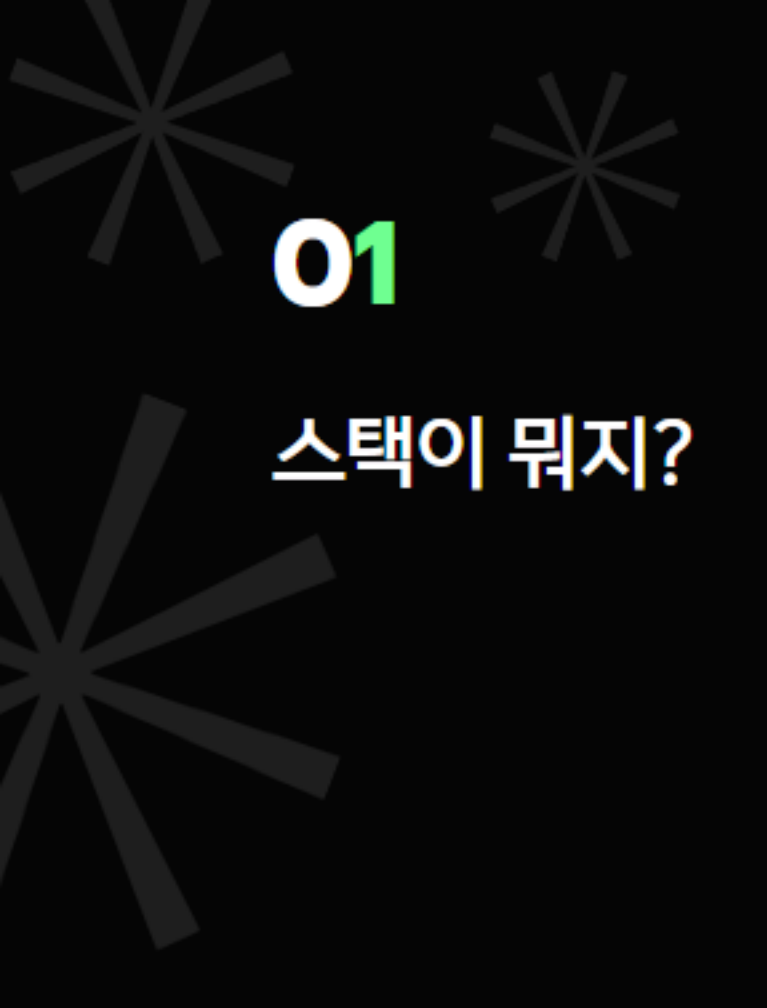
주소 $\rightarrow$ 값

0 $\rightarrow$ 3

1 $\rightarrow$ 4

2 $\rightarrow$ 5

st 내의 2번째 원소를 찾기 위해서는
st[0]의 주소 + 2를 하면 바로 접근할 수 있겠죠?



01

스택이 뭐지?

$st[i]$ 의 주소는 $st[0] + i$

원소를 연속된 메모리 공간에 배치하면 스택 내의 i 번째 원소를 찾을 때 $O(1)$ 에 접근할 수 있는 이점이 있습니다.



01

스택이 뭐지?

그래서 문제가 생기는 지점이 있습니다.
`st = [0, 1, 2, 3, 4, 5, 6, 7, 8]` 일 때
맨 앞에 있는 0을 제거해 봅시다.



01

스택이 뭐지?

주소 -> 값

0 -> 0

1 -> 1

2 -> 2

3 -> 3

4 -> 4

...



01

스택이 뭐지?

주소 $\rightarrow$ 값

0 $\rightarrow$ (-)

1 $\rightarrow$ 1

2 $\rightarrow$ 2

3 $\rightarrow$ 3

4 $\rightarrow$ 4

...



01

스택이 뭐지?

맨 앞에 있는 0이 제거되었지만,
스택의 시작 주소를 유지해야 하기 때문에
뒤에 있는 값들을 하나하나 앞으로 옮겨줘야 합니다.



01

스택이 뭐지?

주소 -> 값

0 -> 1

1 -> 2

2 -> 3

3 -> 4

4 -> 5

...

이 작업을 수행하는 데에는 $O(N)$ 의 시간이 걸립니다.

(N은 앞으로 끌어와야 하는 원소의 수)

꼭 맨 앞이 아니더라도 스택의 중간에서 원소를 빼게 되면

연속성 유지를 위해 빠진 원소의 뒤에 있던 원소들을

$O(N)$ 을 들여 옮겨와야 합니다.

하지만 맨 뒤에 있는 원소를 삭제한다면
원소들을 끌어올 필요가 없기 때문에 $O(1)$ 로 작업을 완료할 수 있습니다.
이것이 `pop()`이고,
스택에서는 주로 `append()`, `pop()`만을 이용합니다.



01

스택이 뭐지?

예제: 스택 수열 (#1874)

반드시 오름차순으로만 push(append)해야 하는데,
1~n 범위의 수가 n개 주어지고, 중복되는 수는 없으므로
1, 2, 3, 4, 5, ..., n의 순서로 스택에 넣으면 된다.



01

스택이 뭐지?

한번에 수를 다 넣지 말고, 넣는 중간에 수들을 빼면서
주어진 순서대로 뽑는 방법을 생각해 보자.



01

스택이 뭐지?

만약 1~ k 까지의 수를 스택에 넣은 상황에서 k 보다 큰 n 을 빼야 한다면
오름차순을 지켜야 하니 $k+1 \sim n$ 까지의 수를 스택에 push해야 한다.

반대로 1~ k 까지의 수를 스택에 넣은 상황에서 k 보다 작은 n 을 빼야 한다면
반드시 스택의 맨 위에 있는 원소가 n 이어야 한다.
만약 그렇지 않다면 주어진 수열을 만드는 것은 불가능하다.

예제 입력 1에서

가장 먼저 pop해야 하는 원소는 4이다.

시작 지점에서 스택은 비어 있으므로 1~4의 원소를 push한 뒤

pop해주면 맨 처음 원소로 4를 꺼낼 수 있다.

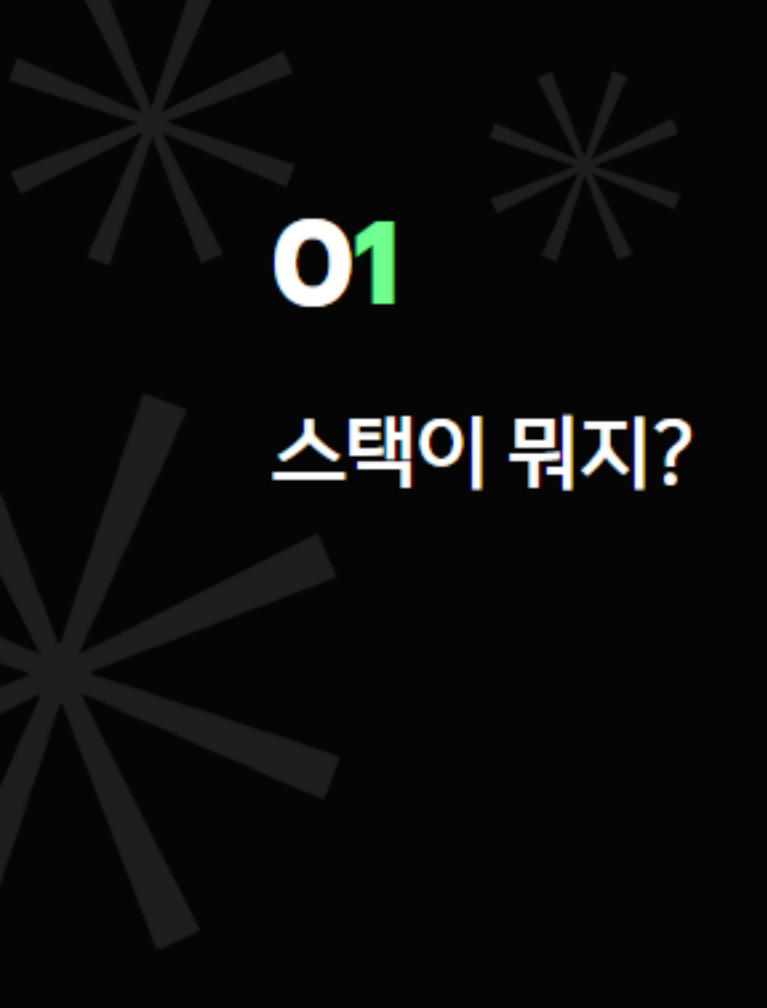
$st = [1, 2, 3, 4] \rightarrow st = [1, 2, 3]$



01

스택이 뭐지?

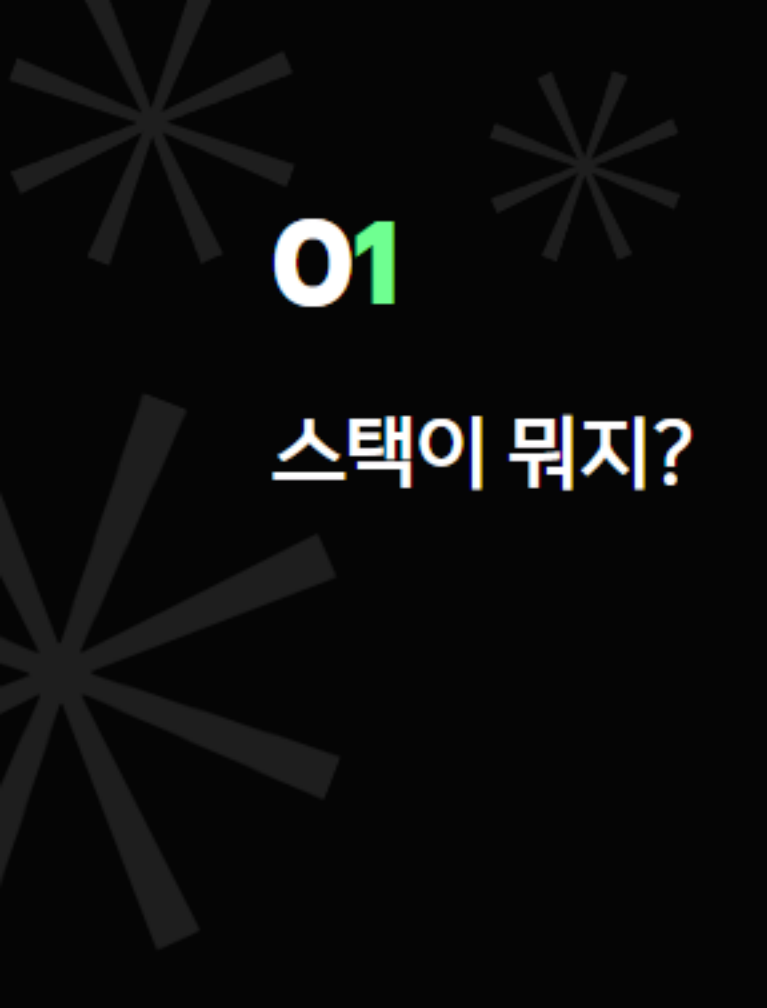
다음으로 꺼내야 하는 원소는 3인데,
현재 $st = [1, 2, 3]$ 이므로 바로 pop해준다.



01

스택이 뭐지?

이제 6을 pop해야 하는데,
현재까지 사용한 수는 1~4이므로 5, 6을 push한 뒤 pop한다.
 $st = [1, 2] \rightarrow st = [1, 2, 5, 6] \rightarrow st = [1, 2, 5]$



01

스택이 뭐지?

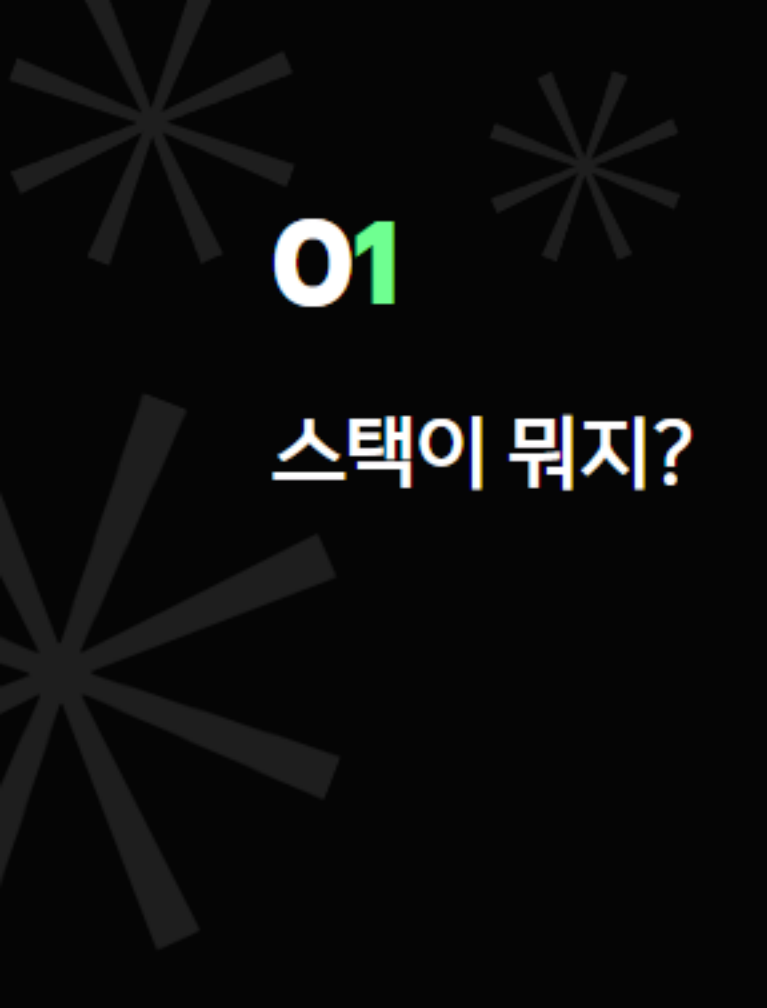
다음으로 꺼낼 8도 현재까지 사용한 1~6보다 크므로 7, 8을 push
 $st = [1, 2, 5, 7, 8] \rightarrow st = [1, 2, 5, 7]$



01

스택이 뭐지?

나머지 7, 5, 2, 1은 순서대로 pop해주면 된다.
최종적으로 예제 출력 1이 나오게 된다.



01

스택이 뭐지?

예제 입력 2에서

1, 2, 5까지 진행했을 때 스택의 상태는

[3, 4]

하지만 다음으로 pop해야 하는 원소가 3이므로 불가능하다.

01

스택이 뭐지?

```
def solve():
    n = int(input().strip())
    stack = []
    nums = []
    res = []
    for _ in range(n):
        nums.append(int(input().strip()))
    counter = 0 # 현재까지 사용한 수
    for tmp in nums:
        while counter < tmp: # 아직 사용하지 않은 수일 경우 push될때까지 while문이 돌아감
            counter += 1
            stack.append(counter)
            res.append("+")

        if stack[-1] == tmp: # pop이 가능한 경우
            stack.pop()
            res.append("-")
        else: # 불가능할 경우 종료
            print("NO")
            return
    for i in res:
        print(i)
    return
```

참고사항:

이해를 돕기 위해 원소 한개마다 메모리를 1바이트씩만 할당했지만,
실제 파이썬에서 리스트 안에 들어가는 값은
객체 자체가 아니라 객체의 메모리 주소(포인터)입니다.

왜냐하면 다른 언어의 배열과 달리 리스트는 다양한 자료형을 한꺼번에 저장할 수 있고,
이때 임의접근을 위해서는 각 자료의 크기를 맞춰야 하므로 진짜 객체 대신 객체의 포인터를 사용합니다.
(객체는 다른 곳에 저장)

따라서 64비트 환경에서 메모리 주소는 8비트이고,
원소의 주소는 0, 1, 2, 3, 4...가 아닌 0, 8, 16, 24, 32... 로 늘어나게 됩니다.
주소값 계산은 $st[i] = st[0] + 8 * i$



—



놀이공원에서 줄을 서 있는 상황을 생각해 보자.
아마 맨 앞에 서 있는 사람이 가장 먼저 놀이기구를 탈 것이다.
맨 앞에 서 있다는 것은
현재 줄 내에서 가장 빠르게 줄을 선 사람이라는 말과 같다.

FIFO(First In, First Out)

큐에서는 줄을 선 사람들처럼
가장 먼저 들어간 원소가 가장 빨리 나온다.
이때 시간복잡도는 $O(1)$ 이다.


```
qu = []
```

```
qu.append(1) -> [1]
```

```
qu.append(2) -> [1, 2]
```

```
qu.append(3) -> [1, 2, 3]
```

```
qu.pop() -> [2, 3]
```

```
qu.pop() -> [3]
```

덱이란?

큐와 스택을 합쳐놓은 자료구조로,
앞에서 빼는 것도 $O(1)$, 뒤에서 빼는 것도 $O(1)$ 에 해결할 수 있다.



02

큐

파이썬에서 사용하기 위해서는
`from collections import deque`

`qu = deque()`

일반적으로 스택이 필요할 때는 리스트,
큐나덱이 필요할 때는 덱을 사용한다.

그럼 그냥 덱을 쓰지 뭐하러 스택과 덱을 나눠놨을까?

- 덱은 리스트에 비해 메모리 사용량이 많다.
- 이중 연결 리스트 기반이라 []로 임의접근이 불가능하다.
(정확히는 조금 다르긴 하지만)

이중 연결 리스트란?

리스트처럼 연속된 메모리 공간에 배치하는 것이 아니라
띄엄띄엄 있지만 앞뒤의 메모리 주소를 기억하고 있어서
순차적으로 접근해야 하는 자료구조

〈덱 결론〉

왼쪽 삽입: `appendleft()`, $O(1)$

오른쪽 삽입: `append()`, $O(1)$

왼쪽 삭제: `popleft()`, $O(1)$

오른쪽 삭제: `pop()`, $O(1)$

원소 접근: `deque[N]`, $O(N)$

〈덱, 큐 결론〉

왼쪽 삽입: `appendleft()`, $O(1)$

오른쪽 삽입: `append()`, $O(1)$

왼쪽 삭제: `popleft()`, $O(1)$

오른쪽 삭제: `pop()`, $O(1)$

원소 접근: `deque[N]`, $O(N)$

〈스택 결론〉

삽입: `append()`, $O(1)$

삭제: `pop()`, $O(1)$

원소 접근: `stack[N]`, $O(1)$



02

큐

예제: 카드2(#2164)



02

큐

N이 좀 크긴 하지만
덱을 이용하면 $O(N)$ 에 해결할 수 있다.



02

큐

$N = 6$

$qu = [1, 2, 3, 4, 5, 6]$



02

큐

qu = [1, 2, 3, 4, 5, 6]

qu.popleft() → qu = [2, 3, 4, 5, 6]



02

큐

`qu = [2, 3, 4, 5, 6]`

`qu.append(qu.popleft()) → qu = [3, 4, 5, 6, 2]`



02

큐

qu = [2, 3, 4, 5, 6]

qu.append(qu.popleft()) → qu = [3, 4, 5, 6, 2]

이 과정을 1회 반복할 때마다 원소의 수가 1씩 줄어들게 됨
원소의 수가 1이 될 때까지 반복함
즉 전체 시간복잡도는 $O(N)$

```
import sys
input = sys.stdin.readline
from collections import deque

def solve():
    n = int(input().strip())
    qu = deque(i for i in range(1, n+1))
    while len(qu) > 1:
        qu.popleft()
        qu.append(qu.popleft())
    print(qu[0])

solve()
```


자주 쓰이지는 않지만 덱에는 `rotate(n)`라는 메소드도 있다.
다이얼을 돌리듯이 덱에 있는 원소를 **오른쪽**으로 순환시킬 수 있다.

(n 이 음수일 경우 왼쪽)

시간복잡도는 $O(n)$

맨 앞에 있는 원소를 빼서 맨 뒤에 넣는 연산은 **왼쪽으로 한칸 돌리는 것과 같다.**

```
import sys
input = sys.stdin.readline
from collections import deque

def solve():
    n = int(input().strip())
    qu = deque(i for i in range(1, n+1))
    while len(qu) > 1:
        qu.popleft()
        qu.rotate(-1)
    print(qu[0])

solve()
```


우선순위

—



03

우선순위 큐는

우선순위가 가장 높은 원소부터 빠지는 자료구조이다.

사용하기 위해서는
import heapq 또는

from heapq import *

이러면 매번 앞에 heapq.를 붙이는 수고를 덜 수 있다.

따로 자료구조를 선언해야 하는 건 아니고,
heapq 모듈을 이용하면 리스트를 우선순위 큐처럼 활용할 수 있다.

삽입: `heappush((삽입할 리스트), (삽입할 원소))`, $O(\log N)$

삭제: `heappop((삽입할 리스트), (삽입할 원소))`, $O(\log N)$

최솟값 확인: `(리스트)[0]`, $O(1)$

`heappush(hq, 1)`

`heapq.heappush(hq, (1, 2, 3))`

`heappop(hq)`

와우! 마법같은 자료구조인데 원리가 뭐지?

원리는... 지금은 몰라도 된다!

아직 그래프 탐색조차 배우지 않은 상태이기 때문에...

그리고 모르고 써도 사실 상관은 없다. 시간복잡도가 $\log$ 라는 것만 기억하자.

지금은 몰라도 어차피 **자료구조** 과목에서 배우게 된다...

지금부터 사용할 heapq는 **최소 힙**이라는 자료구조이다.
리스트 안에 있는 원소들을 **사전식**으로 비교해서
push는 $O(\log N)$,
pop()을 할 때는 가장 **작은 원소**부터 $O(\log N)$ 시간에 꺼내준다.



03

문제 풀어보기

예제: 최소 힙(#1927)

최소 힙의 가장 기본적인 문제이다.
따로 구현할 필요는 없고, heapq 모듈의 사용법을 배워보자.

qu = []

push할 때: `heappush(qu, (값))`

pop할 때: `heappop(qu)`

주의사항: `heapq`를 사용하다가 일반 `append`나 `pop`을 사용하면
힙 속성이 깨진다.

`heapify(lst)`를 해줘야 다시 힙처럼 사용할 수 있다.

```
import sys
from heapq import heappush, heappop
input = sys.stdin.readline

def solve():
    N = int(input())
    qu = []
    for _ in range(N):
        x = int(input())
        if x != 0:
            heappush(qu, x)
        elif x == 0 and qu:
            print(heappop(qu))
        else:
            print(0)

solve()
```



03

문제 풀어보기

예제: 크리스마스 선물 (#14235)

선물을 주는 도중에 선물을 충전하고,
그때그때 가장 좋은 선물을 줘야 한다.
만약 선물을 줄 때마다 현재 선물 중에 최댓값을 찾는다면
 $O(n*an)$ 이 돼서 시간초과가 날 것이다.

하지만 우선순위 큐로 처리한다면
 $O(n \cdot \log(a_n))$ 로 줄일 수 있다.

흐음..그런데 문제가 있다.

heapq의 기본 설정은 **최소 힙**이라서
pop할 때는 항상 리스트 내에서 **가장 작은 원소**를 반환하는데,
문제를 풀기 위해서는 **가장 큰 원소**를 반환해야 한다.

어떻게 해야 할까?

가장 쉬운 해답이 있다.

push 할 때 넣는 값에 - 부호를 붙여주면

원래는 가장 크던 값이 가장 작아진다.

그러면 최대 힙으로 사용할 수 있다!

대신, 꺼낼 때도 다시 - 부호를 붙여야 값이 복원된다는 점을 잊지 말자.

$[-3, -1, 4, 5, 7]$ 에 각각 -1 을 곱해주면

$[-7, -5, -4, 1, 3]$ 이 되어서 거꾸로 정렬되는 것과 같은 효과를 가져온다.


```
import sys
from heapq import *
 = sys.stdin.readline

n = int(input().strip())
hq = []
for _ in range(n):
    a, *s = map(int, input().split()) # 입력받는 시점에서 s의 길이를 모르므로 *처리
    if a == 0:
        if hq:
            print(-heappop(hq)) # 다시 - 부호를 붙여서 출력
        else:
            print(-1)
    else:
        for i in s:
            heappush(hq, -i) # - 부호를 붙여서 힙에 저장
```

문제에 맞는 알고리즘을 설계하기 위해서는
적절한 자료구조를 선택하는 것이 중요하다!

큐, 스택, 덱, 우선순위 큐

각각이 하는 일이 복잡하진 않지만 앞으로 ps를 하는 데 꼭 필요한 것들이니
반드시 기억하도록 하자!



강의 설문조사

QR 제공 예정

Thank you For Listening

6주차 스터디를 들어주셔서 감사합니다.
뒷풀이에 참여하실 분들만 강의실에 남아주세요!

—

